

⇒ Convolutional Neural Network (CNN)

1) ANN → Supervised Learning

- Classification
- Regression

2) CNN ⇒ Input is in form of
images

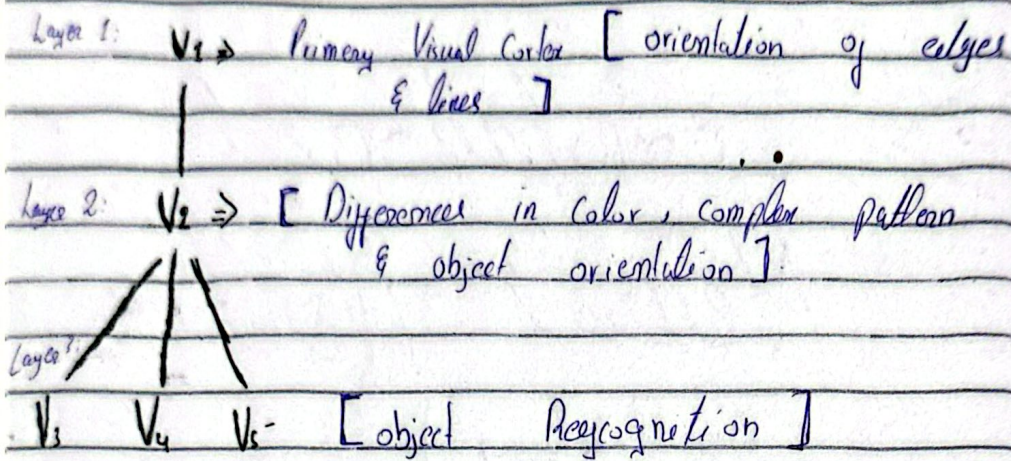
e.g. Image classification
object detection, segmentation

x — x

⇒ Cerebral Cortex & Visual Cortex:

⇒ Visual Cortex: Region of brain that
receives, integrates & processes visual
info from retina

Visual Cortex has 5 layers
 $V_1 - V_5$



* Visualize image after info from Retina is passed through all three layers.

* CNN tries to mimic this with each layer capturing some in one layer

X ————— X

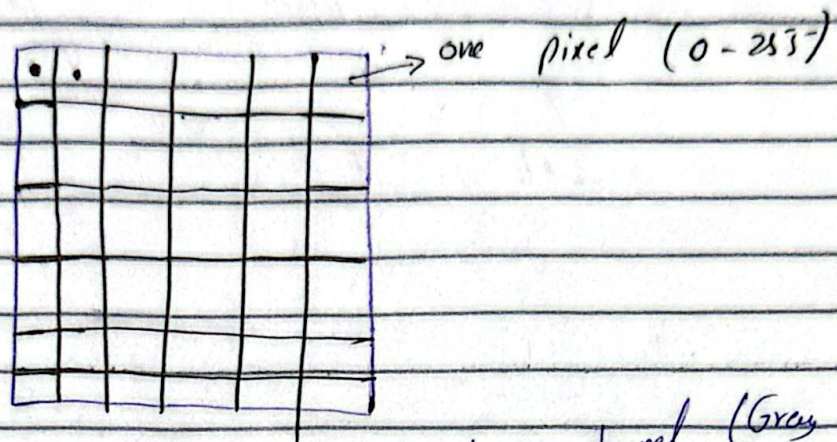
↳ Images:

↳ RGB Images:

→ Each image is formed/combination of Three colors/channels → Red, Green, Blue.

Images are represented in Pixels & each pixel value ranges from 0 - 255

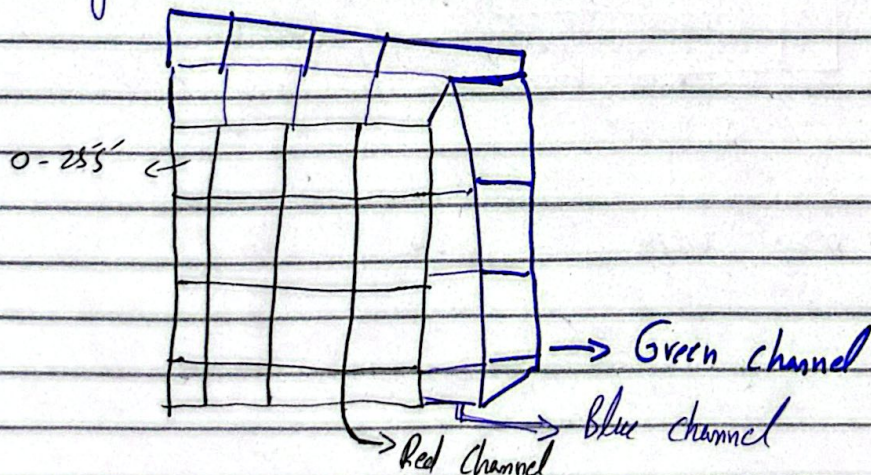
6x6 Pixel Image



6x6 x 1 means only one channel (Gray Scale Image)

⇒ In Gray scale Image There is no RGB & only Gray color.

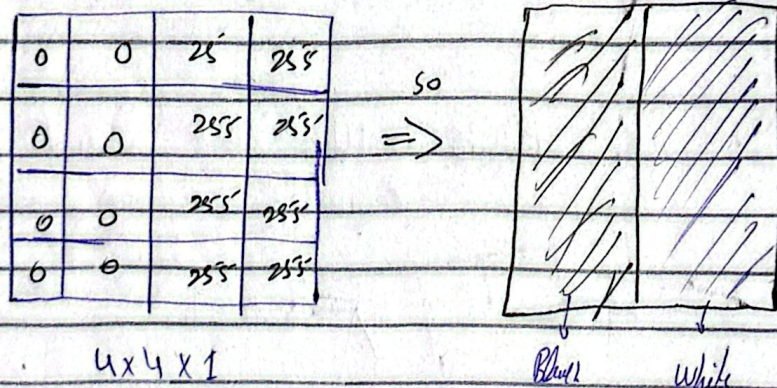
* For RGB we have Grid (Pixel grid) for each channel.



4x4 x 3

⇒ Convolutional Operations In CNN

* Convolutional operation process image & extract some information about image (just like $V_1 - V_5$ each layer extract some info).



⇒ 0 means Black, 255 means White

* observe There's a margin b/w black & white part & This layer extract that info.

⇒ But There are Some
Steps to follow in
This Branch Operation.

↳ Step 1: Normalize the pixel $\Rightarrow (0,1)$

* $\div$ every pixel by 255 $\left\{ \begin{array}{l} \frac{0}{255} = 0 \\ \frac{255}{255} = 1 \end{array} \right.$

0	0	1	1
0	0	1	1
0	0	1	1
0	0	1	1

$4 \times 4 \times 1$

↳ Step 2: Convolution Operation

* Multiply Normalized pixel with a filter to extract info.

0	0	1	1
0	0	1	1
0	0	1	1
0	0	1	1

*

1	0
1	0

filter

$\Rightarrow$

0	-4	-4	0
0	-4	-4	0
0	-4	-4	0
0	-4	-4	0

So How This Multiplication occurs?

we place the filter on pixel & multiply

* we place the filter (2x2) on pixel & Multiply.

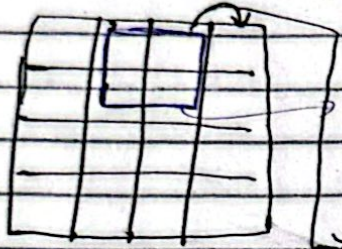
e.g



placed filter here & Multiply

* when multiplication finishes we take a jump on one step called stride

→ jump here after finish



Now Multiply This & when finishes the jump

→ jump here when finishes



Example

0	0	0	1	1	1
0	0	0	1	1	1
0	0	0	1	1	1
0	0	0	1	1	1
0	0	0	1	1	1
0	0	0	1	1	1

1	0	-1
2	0	-2
1	0	-1

3x3 filter

6x6x1

e.g filter is here.

0	0	0	1	1	1
0	0	0	1	1	1
0	0	0	1	1	1
0	0	0	1	1	1
0	0	0	1	1	1
0	0	0	1	1	1

$$\begin{aligned}
 & \therefore (1 \times 0) + (0 \times 0) + (-1) \times 1 + \\
 & (2 \times 0) + (0 \times 0) + (-2) \times 1 + \\
 & (1 \times 0) + (0 \times 0) + (-1) \times 1 = -4
 \end{aligned}$$

So this entire region will become -4

→ Vertical edge filter $\Rightarrow$ This filter finds Vertical edges
(just like in human Brain we find edges).

* There are many filter e.g horizontal edge filter etc

0	-4	-4	-4	1	0
0	-4	-4	-4	1	0
0	-4	-4	-4	1	0
0	0	0	1	1	1
0	0	0	1	1	1
0	0	0	2	1	1

That whole region becomes
-4.

x ——— x

So when Multiplication occurs will get 4×4 output

0	0	0	1	1	1
0	0	6	1	1	1
0	0	6	1	1	1
0	0	6	1	1	1
0	0	0	1	1	1
0	0	0	1	1	1

$6 \times 6 \times 1$

1	0	-1
1	0	-1
1	0	-1

3×3

filter

0	-4	-4	0
0	-4	-4	0
0	-4	-4	0
0	-4	-4	0

4×4

output

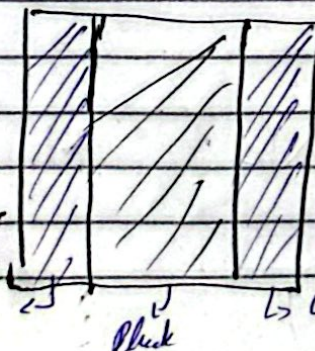
Step 3: Denormal The Output.

* Minimum Value (-4)
becomes 255

* Max Value (0) becomes (255)

255	0	0	255
255	0	0	255
255	0	0	255
255	0	0	255

$\Rightarrow$



*** This filter found Vertical
edges. (vertical edge filter)

White

Black

White

⇒ We can have any number of filters & based on that we will get that many outputs.

↳ Output Matrix dimension formula

* $\text{Original} - \text{filter} + 1$

$6 - 3 + 1 = 4 \Rightarrow$ & That's why we get 4×4 output.

• But - Reducing $6 \times 6 \rightarrow 4 \times 4$ means we are losing quality.

This is solved using padding.

X — X

Padding:

* When we give 6×6 , we should get output of 6×6 too.
not 4×4

* This is achieved using padding.

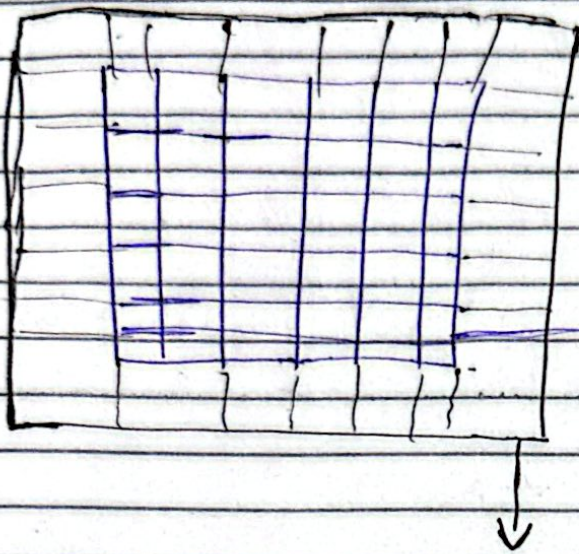
⇒ We add Cusion:

↳ an extra layer



→ 0.6

→ one layer of padding



→ original 6x6

⇒ We added one layer of padding

Now it became 3x3

⇒ Now we multiply with filter
we will get 6x6.

→ Padding Layer Formula

$$\boxed{\text{original} - \text{filter} + 2p + 1}$$

Now in output I want 6x6 so

$$6 - 3 + 2p + 1 = 6$$

$$3 + 2p + 1 = 6$$

$$2p = 6 - 4$$

$$2p = 2$$

$$\boxed{p = \frac{2}{2} \Rightarrow 1}$$

★ To get 6x6 output

I would add 1-layer
of padding.

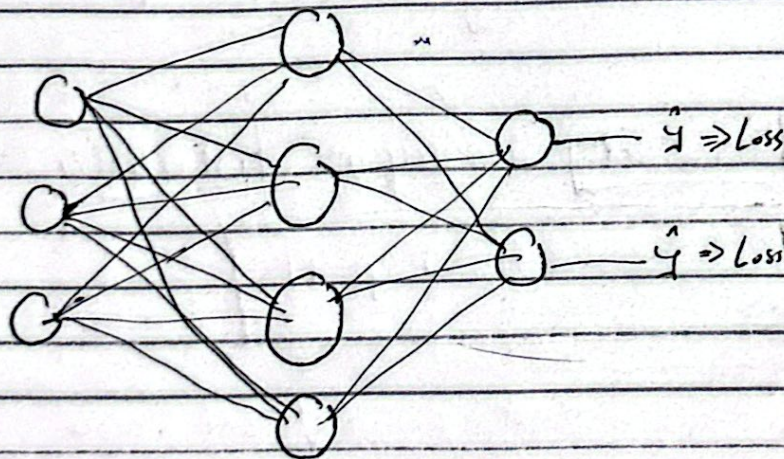
⇒ Now what should be the values in the boxes created VTA packing.

(1) Zero-packing ⇒ fill all with zero

(2) Neighbor packing ⇒ fill with neighbor value

x — x

⇒ Operations of CNN vs RNN



1st Operation ⇒ $z = w^T x_i + b$

2nd Operation ⇒ Activation (z)

3rd " ⇒ Loss (y - $\hat{y}$)

4th " ⇒ BP & weight updation.

CNN is based in Visual Cortex ($V_1 - V_5$)

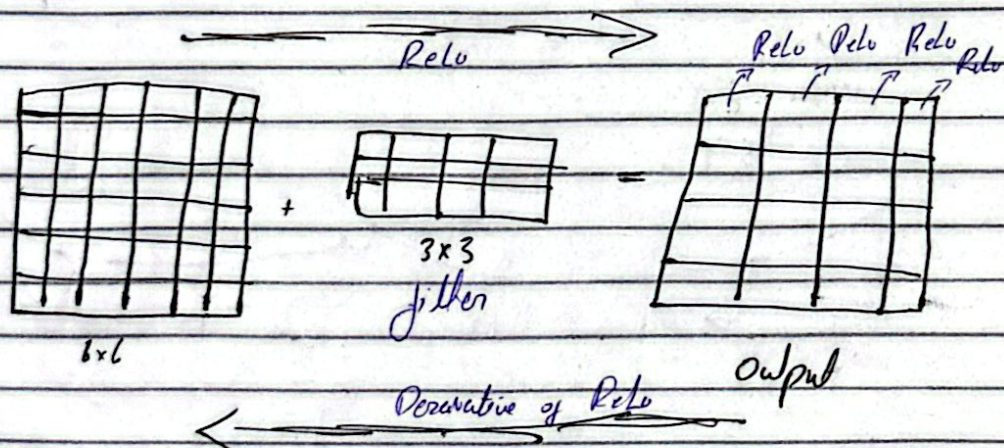
* We use different filters to find some pattern, edges, shapes.

* We can have any number of filter

* The filters are randomly initialized.

* We update the filters in forward & backward propagation

* We perform ReLU operation of output matrix each & every value.



* ReLU $\max(0, x) \rightarrow$ FP

* Derivative of ReLU either 0 or 1 $\rightarrow$ BP

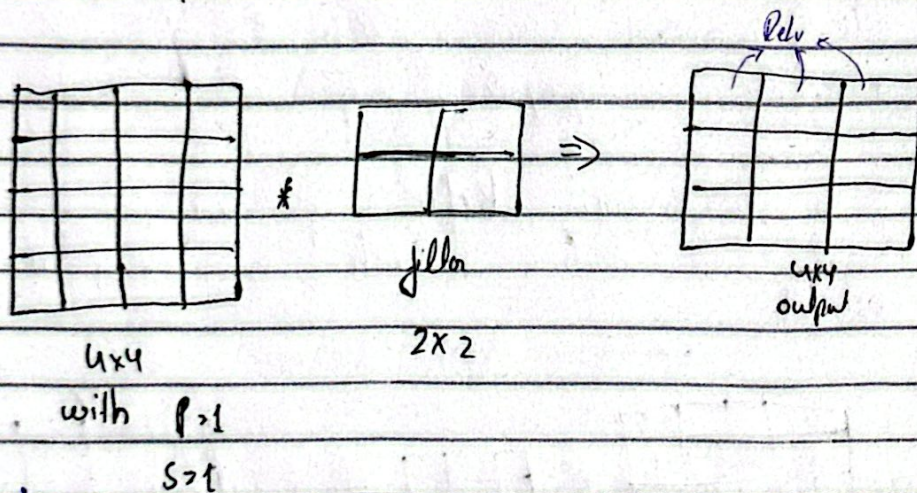
$\Rightarrow$ We update weights in filter

1st Operation $\Rightarrow$ Convolutional operation
2nd Operation $\Rightarrow$ Apply ReLU

$\Rightarrow$ This is called Convolution Layer.

$\Rightarrow$ Max-pooling, Min pooling, Avg/Max Pooling:

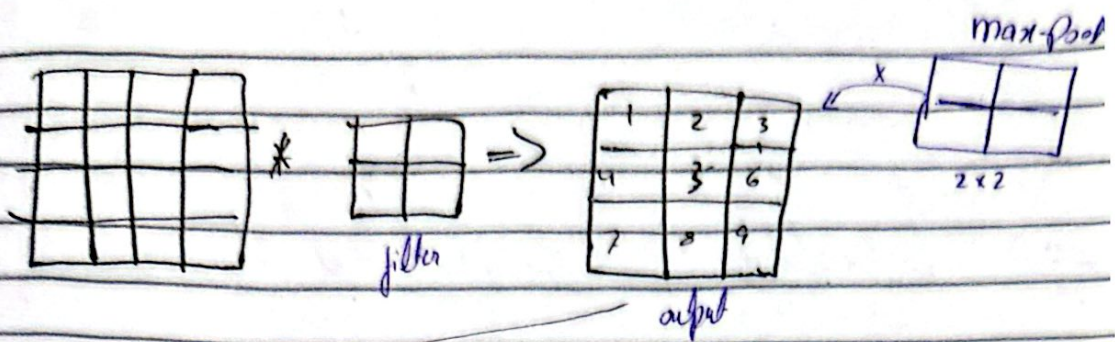
$\Rightarrow$ Pooling ≥ 1 & Stride > 1



$\Rightarrow$ This entire layer is called Convolutional operation layer

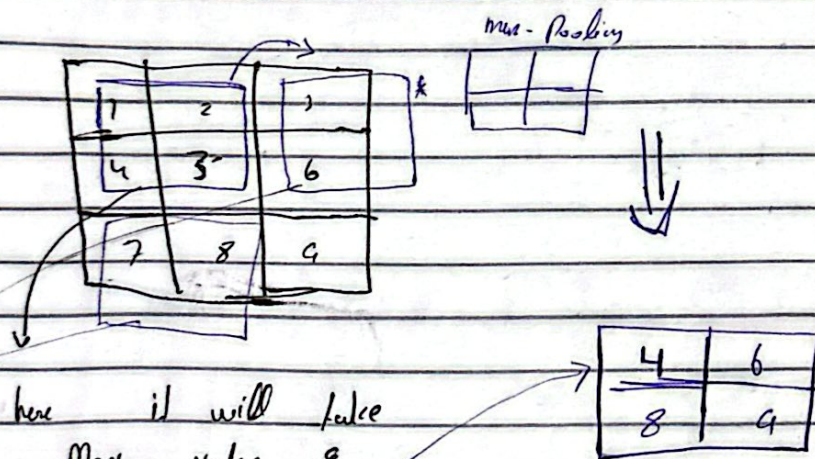
* Now after Convolution layer (Convolution operation + ReLU operation)
I apply max-pooling on output.

$\Rightarrow$ i.e. Apply a filter (matrix) as Max pool on output. (output $\times$ max pool filter)



* In simple words $\Rightarrow$ Max pool extracts the enhanced output.

For Cam $\Rightarrow$ In Input Image The initial filter extracts/focuses wheels of car, So max pool will just focus on car wheel & extract that specific part.



$\Rightarrow$ In here it will take the Max value & put in output (4)

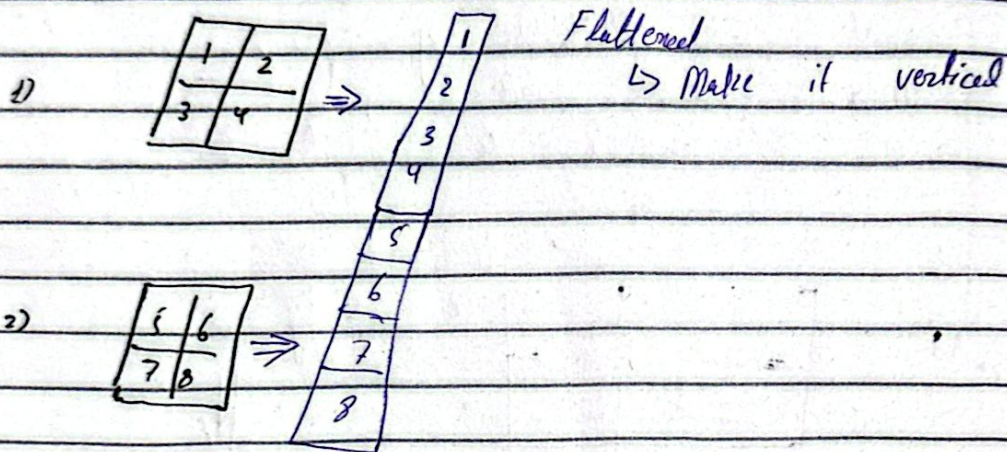
$\Rightarrow$ Then jump by Stride = 2

$\Rightarrow$ Here Max is 6

$\Rightarrow$ Max = 9

⇒ Fully Connected Layer IN CNN

⇒ Flattened: Applied on output of Pooling



⇒ Fully connected layer ⇒ Connected Flattened with neurons

